Biased Random Walk based Web API Recommendation in Heterogeneous Network

Xiaocao Hu
School of General Education
Dalian University of Technology
Panjin, China
huxiaocao@dlut.edu.cn

2024 IEEE International Conference on Web Services (ICWS) | 979-8-3503-6855-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICWS62655.2024.00038

***Abstract*—Benefit from the remarkable development of cloud computing, massive Web APIs have been published and numerous innovative mashups have emerged on the Internet. The rapid increase of available Web APIs brings a significant challenge on how to discover suitable APIs for mashup creation. Various approaches have been proposed to recommend Web APIs, which mainly focus on employing collaborative filtering techniques such as matrix factorization and deep learning techniques such as graph neural network. However, existing studies suffer from a few limitations: collaborative filtering based methods are not able to capture the high-order connectivity information between APIs and mashups, and deep learning based methods are not expressive enough to capture the diversity of connectivity patterns due to the simple graph structure. To tackle the limitations, we proposes a biased random walk based Web API recommendation approach. We first construct a heterogeneous network to model different types of connectivity. And then we design a strategy in terms of neighbor nodes' types to guide the biased random walk and to capture diversity of high-order connectivity patterns. Finally we generate candidate Web APIs for mashups based on their embedding vectors. Experiments conducted on a real-world dataset show that our approach improves in precision by 13.89%, in recall by 17.16%, in F1 by 15.86%, and in NDCG by 2.85%.**

***Index Terms*—Web API Recommendation, Mashup Creation, High-order Connectivity, Biased Random Walk, Heterogeneous Network**

I. Introduction

Benefit from the remarkable development of cloud computing, Web APIs (application programming interfaces) allow users to remotely access the data or functionality in a programmable way. Driven by the API economy [1, 2], more and more Web APIs have been published on the Internet. As various complex user requirements emerge, functionalities of existing Web APIs have become insufficient to meet complex needs. Regarding this concern, numerous mashups have been created and developed by composing several existing Web APIs [3]. However, with the rapid increasing number and diversity of Web APIs, mashup developers have to face the huge challenge of finding appropriate Web APIs from the massive candidates. Therefore, there is an urgent need to recommend relevant Web APIs for mashup creation.

Various approaches have been proposed to recommend Web APIs for mashup creation. Collaborative filtering is a widely used technique in previous work, which can be generally categorized into memory-based [4] and model-based [5–7]. Memory-based methods usually perform badly when the invocation data is sparse. To deal with the data sparsity problem, model-based methods incorporate auxiliary information, such as descriptions and tags, into the model. However, they are not able to capture the high-order connectivity information between APIs and mashups. Recently, deep learning is introduced into API recommendation to address this issue in some emerging work [8–12]. However, the graph structures in these methods are rather simple, mainly relying on mashup-API invocations, while various types of connectivity exist in real-world network. For example, as for two APIs, a similarity relation exists when they have similar functionalities, a co-occurrence relation exists when they are invoked in the same mashup, a tagging relation exists when they are labeled with the same tag. All these relations contribute to the representation learning, and the flexibility in exploring different kinds of neighborhoods is the key to improve the embedding quality.

To tackle the above limitations, we propose a biased random walk based method by exploring the heterogeneous network of APIs and mashups. More specifically, we firstly construct a heterogeneous network by leveraging the mashup-API invocation records, descriptions and attributes of APIs and mashups. And then we design a strategy to guide the biased random walk, which efficiently explore diverse neighborhoods of a given node, thus capturing diversity of high-order connectivity patterns. Experiments are conducted on a real-world dataset, and experimental results show that our approach performs better than baseline methods, which indicates that the biased random walk strategy has positive effect on improving the recommendation performance.

In summary, the main contributions of this paper include:

- We construct a heterogeneous network to represent different types of connectivity and design a biased random walk strategy for efficiently exploring diverse neighborhoods to augment the representation.
- We utilize a real-world dataset to conduct comprehensive experiments, and the performance comparison with baseline methods demonstrates the effectiveness of our approach.

The remainder of this paper is organized as follows: Section II introduces the related work, Section III provides details of our approach by presenting the heterogeneous network

2836-3868/24/$31.00 ©2024 IEEE
DOI 10.1109/ICWS62655.2024.00038

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.

and the biased random walk strategy, Section IV reports the experimental results, Section V concludes this paper.

II. Related Work

API recommendation has been an active research area for years and various approaches have been proposed. Existing studies for API recommendation can be roughly categorized into collaborative filtering based and deep learning based.

Collaborative filtering is widely used in recommendation, which can be generally classified into memory-based and model-based methods. Cao [4] utilizes the history records to compute the content similarity and combines the user interest value and the prediction value. However, memory-based methods always suffer from the data sparsity problem. Zhong [5] reconstructs profiles of services by applying the author topic model and exploits features of words on the basis of reconstructed profiles. Fletcher [6] explores interactions between users to learn latent representations of users and then incorporates user representation into the matrix factorization model. Botangen [7] considers geographic locations as a significant contextual factor and integrates the geographical relevance into the matrix factorization model. Benefit from discovering the latent factors, model-based methods perform better than memory-based methods. However, model-based methods cannot model the high-order connectivity information.

Recently, deep learning has attracted researchers' attention and has been introduced into API recommendation. Lian [8] utilizes the neural graph collaborative filtering [13] to capture more semantic information from mashup-API invocations. Gong [9] models the recommendation as a graph search problem and finds the minimal group Steiner trees based on API-API co-occurrences. Wang [10] exploits the knowledge graph to model mashup-API invocations and performs random walk with restart according to the knowledge graph. Wang [11] employs the convolutional neural network to capture the functional semantics and applies the multi-layer graph neural network to capture structures. Mai [12] incorporates the hypergraph neural network with multiple API attributes and identifies neighbors of different importance with a hyperedge and node specific attention mechanism. Due to the ability of capturing non-linear features, deep learning based methods are able to model the high-order connectivity information. However, most of these methods solely rely on single type of connectivity, mainly on mashup-API invocations, thus not expressive enough to capture the diversity of connectivity patterns in heterogeneous networks.

Different from previous studies, our proposed approach can not only capture the high-order connectivity information, but also explore various connectivity patterns to augment representations of APIs and mashups.

III. Methodology

We first present concepts in the heterogeneous network of APIs and mashups, and then introduce the biased random walk strategy.

A. Network Construction

A mashup is the composition of one or more Web APIs, and each mashup/API has its own functionality and attributes. Fig. 1 illustrates the heterogeneous network of APIs and mashups, which comprises three types of nodes and three types of edges.

Mashup-API invocations. Let $A = \{a_1, a_2, ..., a_{|A|}\}$ be the set of APIs and $M = \{m_1, m_2, ..., m_{|M|}\}$ be the set of mashups. An edge exists between an API a_i and a mashup m_j when a_i is invoked by m_j.

Service attributes. Let $T = \{t_1, t_2, ..., t_{|T|}\}$ be the collection of attributes. Attributes can be tags, providers, or protocols and so on. Here we use tags as attributes, since tags used to label a mashup or an API contain rich semantic information. Let $S = M \cup A$. An edge exists between a service s_i and a tag t_j when s_i is labeled with t_j.

Similar services. For an API a_i, its functionality is described by a text description D_i, which is regarded as a bag of words $D_i = \{w_1, w_2, ..., w_{|D_i|}\}$. Given two APIs a_i and a_j, the similarity of a_i and a_j is calculated on the basis of D_i and D_j:

$$sim\left(a_i, a_j\right) = \frac{sim\left(D_i \rightarrow D_j\right) + sim\left(D_j \rightarrow D_i\right)}{2} \tag{1}$$

where, $sim\left(D_i \rightarrow D_j\right)$ is an asymmetric similarity, calculated as a normalized, idf-weighted sum of similarities between words in D_i and the entire bag-of-words D_j [14, 15]:

$$sim\left(D_i \rightarrow D_j\right) = \frac{\sum\limits_{w_a \in D_i} \left(\max\limits_{w_b \in D_j} sim\left(w_a, w_b\right) * idf\left(w_a\right) \right)}{\sum\limits_{w_a \in D_i} idf\left(w_a\right)} \tag{2}$$

where, $sim\left(w_a, w_b\right)$ is the similarity of words w_a and w_b; $\max\limits_{w_b \in D_j} sim\left(w_a, w_b\right)$ denotes the similarity of word w_a and description D_j, which is the maximum similarity of word w_a and any word w_b in D_j; $idf\left(w_a\right)$ represents the inverse document frequency of word w_a, calculated from the corpus of API text descriptions.

Considering that the API-API matrix is rather dense, we utilize a threshold to determine the connectivity of two APIs. An edge exists between APIs a_i and a_j when the similarity of a_i and a_j is greater than the threshold.

Similar to an API, a mashup also has a text description. Consequently, the similarity of mashups m_i and m_j is also calculated based on Eq. (1)-(2). An edge exists between m_i and m_j when the similarity of m_i and m_j is greater than the threshold.

Heterogeneous network. The heterogeneous network of APIs and mashups is denoted by $G = (V, E)$, where $V = M \cup A \cup T$ denotes the set of nodes, and $E = E_i \cup E_s \cup E_a$ indicates the set of edges. E_i, E_s and E_a are the collections of invocation links, similarity links and attribute links respectively. Each edge between nodes u and v is bi-directional and

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.

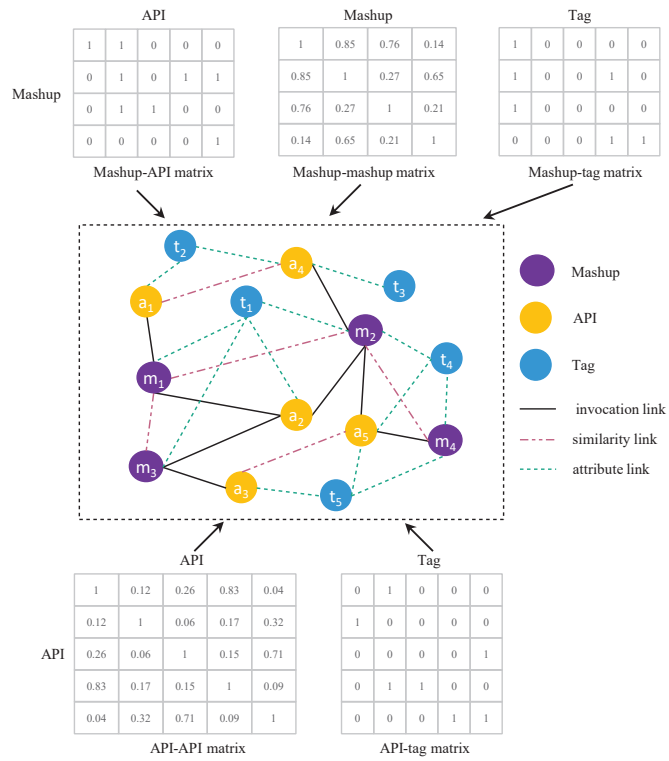


Fig. 1. Heterogeneous network of APIs and mashups

is assigned with a weight w_{uv}:

$$w_{uv} = \begin{cases} 1 & \text{if } (u,v) \in E_i \\ sim(u,v) & \text{if } (u,v) \in E_s \\ 1 & \text{if } (u,v) \in E_a \end{cases} \tag{3}$$

B. Biased Random Walk

Given a source node, a random walk is simulated with a fixed length l and is represented as a sequence of nodes $(n_0, n_1, n_2, ..., n_l)$, in which n_0 is the source node. Formally, node n_i is generated by:

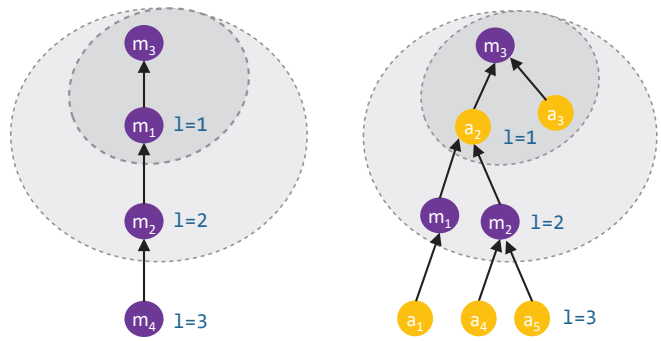
$$P(n_i = v | n_{i-1} = u) = \begin{cases} \frac{\pi_{uv}}{Z} & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where, π_{uv} is the unnormalized transition probability between node u and node v, and Z is the normalizing constant.

Despite the fact that edges are assigned with different weights, it is not sufficient to explore different types of edges in the heterogeneous network. More specifically, a node may be connected to other nodes through three different types of edges. For example, a mashup is connected to an API via the invocation link, to a mashup via the similarity link, and to a tag via the attribute link. These different types of neighbors play different roles in learning the representation of the mashup.

Taking mashup m_3 in Fig. 1 as an example, Fig. 2 shows high-order connectivity for m_3 with two extreme random walk strategies for generating its neighborhood.

- Similarity-based walk. The random walk is represented as a sequence of nodes connected by similarity links. When m_3 is the source code, a random walk of length $l = 3$ tends to be (m_3, m_1, m_2, m_4). As a consequence, the neighborhood of m_3 is restricted to similar mashup nodes, as shown in Fig. 2(a).
- Invocation-based walk. The random walk visits APIs and mashups alternately. Starting from m_3, a random walk of length $l = 3$ is likely to be (m_3, a_2, m_1, a_1), (m_3, a_2, m_2, a_4), or (m_3, a_2, m_2, a_5). As a consequence, the neighborhood is sampled based on invocation links, as shown in Fig. 2(b).



(a) Similarity-based (b) Invocation-based

Fig. 2. High-order connectivity for m_3

These two extreme random walk strategies both play a key role in learning feature representations. For example, the similarity-based walk suggests that m_3 and m_2 have similar functionality at a certain degree. The invocation-based walk indicates the co-occurrence between m_3 and m_2, since they both invoke API a_2.

Inspired by [16], we design a flexible biased random walk strategy to explore different types of neighborhoods and smoothly interpolate between similarity-based walk and invocation-based walk. More specifically, we utilize two parameters φ and ω to guide the random walk and set $\pi_{uv} = \alpha_{\varphi\omega}(u,v) \cdot w_{uv}$:

$$\alpha_{\varphi\omega}(u,v) = \begin{cases} \frac{1}{\varphi} & \text{if } (u,v) \in E_s \\ 1 & \text{if } (u,v) \in E_a \\ \frac{1}{\omega} & \text{if } (u,v) \in E_i \end{cases} \tag{5}$$

Similarity parameter, φ: Parameter φ affects the likelihood of visiting a neighbor node with the same type as u. A small φ would cause the walk to visit more nodes with the same type, thus receiving collaborative signals from high-order neighbors based on the similarity. In contrast, a large φ tends to lessen the effect of similar nodes.

Invocation parameter, ω: Parameter ω controls the probability of visiting a neighbor node connected by the invocation link. If ω is small, the walk is more inclined to visit APIs and mashups alternately, thus capturing signals from high-order neighbors based on the co-occurrence. On the other hand, if ω is large, the walk is less likely to visit nodes through invocation links.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.

C. Feature Learning

Algorithm 1 shows the process of feature learning. We first precompute the transition probabilities π on the basis of edge weights and bias parameters φ and ω (line 1). Considering that each random walk has an implicit bias due to the choice of the source node, we simulate r random walks for each service as the starting node (lines 3-4). For each walk, we sample the next step of the walk based on the transition probabilities (lines 5-10). We treat each random walk as a sentence and apply the Skip-gram model [17, 18] to learn feature representations of nodes in the network (line 15). And then we calculate the inner product based on embeddings of APIs and mashups for recommendation.

Algorithm 1: Feature learning

Input: Heterogeneous network $G = (V, E)$, walk bias parameters φ and ω, random walk length l, walks per node r, window size k, embedding dimension d

Output: Feature representations f

1 $\pi =$ precompute(G, φ, ω);
2 $walks = \{\}$;
3 **for** $i = 1$ *to* r **do**
4 **foreach** $s \in V$ **do**
5 $walk = [s]$;
6 **for** $j = 1$ *to* l **do**
7 $u = walk[-1]$;
8 $N_u =$ getNeighbors(u, G);
9 $v =$ aliasSample(N_u, π);
10 $walk$.append(v);
11 **end**
12 $walks = walks \cup \{walk\}$;
13 **end**
14 **end**
15 $f =$ skip-gram$(walks, k, d)$;
16 **return** f;

IV. Experiments

A. Experimental Setup

Dataset. We construct a real-world dataset to conduct experiments. The dataset, collected from ProgrammableWeb.com, contains 6,434 mashups and 21,832 APIs. Considering that not all mashups invoke APIs and not all API are invoked, we filter out APIs and mashups that have no interaction records, resulting in 47,319 interactions between 5,303 mashups and 898 APIs. Table I shows the basic statistics of the dataset.

Metrics. We adopt several well-known evaluation metrics to measure the performance: Precision@N, Recall@N, F1@N, and NDCG@N. Precision@N is the proportion of correctly recommended APIs in top-N to N, while Recall@N is the proportion of correctly recommended APIs in top-N to truly-used APIs. F1@N is a summary measure that takes both Precision@N and Recall@N into account. NDCG@N is a position-sensitive metric that measures the order of top-N recommended APIs.

TABLE I
Statistics of the dataset

Items	Values
Number of mashups	6434
Number of APIs	21832
Number of interactions	47319
Number of mashups with interactions	5303
Number of APIs invoked by mashups	898
Number of tags	356

Baselines. To verify the effectiveness of our proposed approach, we compare it with the following baseline methods:

- CFrank [4] is a traditional collaborative filtering method.
- MFrank [6] is a matrix factorization method by incorporating the latent representations of users from the user embeddings into matrix factorization.
- RRQrank [10] is a graph-based algorithmic framework by utilizing a knowledge graph and exploiting random walks with restart on it.
- BPRrank [19] is a matrix factorization method by exploiting the maximum posterior estimator based on multiple layer neighborhood.
- NAFM [20, 21] is a hybrid factorization machine model by combining a deep neural network with an attention mechanism.

Hyperparameters. We set the random walk length l to 20, the number of walks r to 80, the window size k to 5, the embedding size d to 128, parameters φ and ω to 0.4 and 0.6 respectively.

B. Experimental Results

1) Performance Comparison: Fig. 3 shows experimental results between our approach BRW (short for Biased Random Walk) and the baseline methods, which compares results of Precision@N, Recall@N, F1@N and NDCG@N as N varies from 1 to 10. It is obvious that our approach achieves better performance than baseline methods in all the four metrics, i.e. improve in precision by 13.89%, in recall by 17.16%, in F1 by 15.86%, and in NDCG by 2.85%.

We observe that MFrank achieves better performance than CFrank, which indicates that learning vector representations of APIs and mashups improves the recommendation performance. BPRrank and NAFM outperform MFrank and CFrank, indicating the benefit of capturing the high-order connectivity. RRQrank is better than MFrank and CFrank in Recall and NDCG, but is worse in Precision and F1 when N is greater than 2. This is probably due to its strategies of adjusting weights of links that heavily depend on the mashup requirements.

The outperformance of our approach can be contributed to the graph structure with diverse edges and the biased walk strategy. By modeling different types of relations into the

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.

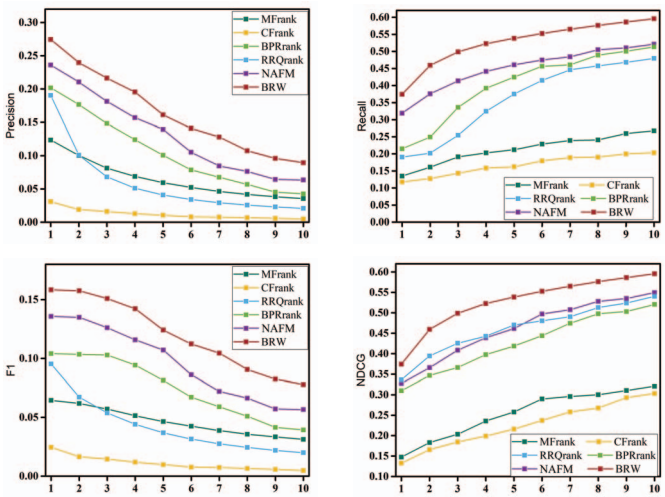


Fig. 3. Performance comparison between BRW and baselines

graph, we can explore diverse neighborhoods of a given node. The biased walk strategy with low φ and ω encourages to explore nodes connected by similarity links and invocation links. As a consequence, we can capture collaborative signals from high-order neighbors based on similarity and co-occurrence.

2) Hyperparameters Analysis: To analyze the effect of hyperparameters, we fix the other parameters and vary only one parameter.

Impact of φ and ω. Parameters φ and ω affect the type of the next node to be visited in the random walk. APIs and mashups are more significant than attributes in learning the representation, edges between services should be assigned with higher weights. Consequently, parameters φ and ω are both set to small values (not greater than 1). To evaluate the impact of these two parameters, we utilize a coarse granularity of grid-search, which is sufficient for achieving close-to-optimal results. Table II shows experimental results with different combinations of φ and ω. We can see that as φ becomes larger and ω becomes smaller, the performance firstly increases and then decrease. This observation is consistent with intuition. If φ is too small, the walk is composed of either APIs or mashups, and edges in the network are reduced to E_s. If ω is too small, the walk is composed of APIs and mashups alternately, and edges in the network are reduced to E_i. In either case, it would cause the over-smoothing problem. In fact, this experiment verifies the effectiveness of the biased random walk strategy.

Impact of l. Parameter l controls the length of a random walk. To analyze the effect of the walk length l, we vary l from 10 to 80, with a step of 10. Fig. 4 presents experimental results with different random walk lengths. We can see that the performance increases at beginning, achieves the best value when $l = 20$, and then decreases as l increases continuously. The reason behind this is that a small l may not be sufficient to capture the high-order connectivity in the network, while a large l may brings a lot of noise information to the representation learning.

TABLE II
IMPACT OF DIFFERENT COMBINATIONS OF (φ, ω)

(φ, ω)	Precision@5	Recall@5	F1@5	NDCG@5
(0.1, 0.9)	0.1091	0.5456	0.1818	0.4667
(0.2, 0.8)	0.1096	0.5480	0.1827	0.4691
(0.3, 0.7)	0.1098	0.5490	0.1830	0.4698
(0.4, 0.6)	**0.1104**	**0.5523**	**0.1840**	**0.4701**
(0.5, 0.5)	0.1092	0.5464	0.1820	0.4661
(0.6, 0.4)	0.1081	0.5418	0.1802	0.4645
(0.7, 0.3)	0.1079	0.5389	0.1800	0.4633
(0.8, 0.2)	0.1078	0.5380	0.1796	0.4610
(0.9, 0.1)	0.1075	0.5326	0.1789	0.4565
(1, 1)	0.1071	0.5340	0.1784	0.4560

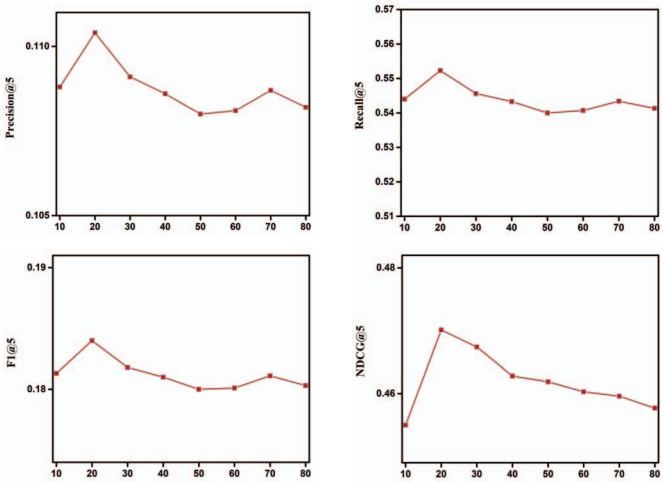


Fig. 4. Impact of different random walk lengths

Impact of d. Parameter d controls the embedding dimensions of APIs and mashups. To analyze the effect of the embedding dimension d, we vary d in values {16, 32, 64, 128, 256}. Table III shows experimental results with different embedding dimensions. We can see that as d increases from 16 to 128 the performance increases significantly, and then slightly decreases when d increases continuously. This observation reveals that it is not always helpful to increase the embedding dimension, since a large dimensionality may result in the increase of noise in representation.

TABLE III
IMPACT OF DIFFERENT EMBEDDING DIMENSIONS

d	Precision@5	Recall@5	F1@5	NDCG@5
16	0.0581	0.2906	0.0968	0.2112
32	0.0933	0.4667	0.1555	0.3732
64	0.1082	0.5409	0.1803	0.4537
128	**0.1104**	**0.5523**	**0.1840**	**0.4701**
256	0.1098	0.5490	0.1830	0.4629

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

In this paper, we propose a biased random walk based Web API recommendation method that explores different kinds of neighborhoods. The benefits are twofold. On one hand, the diverse connectivity information provides the ability to explore different types of neighbor nodes for a given node. On the other hand, the biased random walk strategy allows to make the trade-off between similarity and co-occurrence. Consequently, our approach is capable of capturing the diversity of high-order connectivity patterns in the heterogeneous network. A series of experimental results demonstrate that our approach outperforms the state-of-the-art methods. In the future, we will consider to use hypergraph for API recommendation.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (Grant No. 72271039).

References

[1] W. Tan, Y. Fan, A. Ghoneim, M. A. Hossain, and S. Dustdar, "From the service-oriented architecture to the Web API economy," *IEEE Internet Computing*, vol. 20, no. 4, pp. 64–68, 2016.

[2] X. Guo, H. Zhou, and M. Tang, "The API-mashup ecosystem: A comprehensive study of ProgrammableWeb," *International Journal of Embedded Systems*, vol. 15, no. 2, pp. 132–138, 2022.

[3] S. R. Chowdhury, F. Daniel, and F. Casati, "Recommendation and weaving of reusable mashup model patterns for assisted development," *ACM Transactions on Internet Technology*, vol. 14, no. 2-3, pp. 1–23, 2014.

[4] B. Cao, M. Tang, and X. Huang, "CSCF: A mashup service recommendation approach based on content similarity and collaborative filtering," *International Journal of Grid and Distributed Computing*, vol. 7, no. 2, pp. 163–172, 2014.

[5] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 468–478, 2018.

[6] K. Fletcher, "Regularizing matrix factorization with implicit user preference embeddings for Web API recommendation," in *IEEE International Conference on Services Computing (SCC)*, 2019, pp. 1–8.

[7] K. A. Botangen, J. Yu, Q. Z. Sheng, Y. Han, and S. Yongchareon, "Geographic-aware collaborative filtering for Web service recommendation," *Expert Systems with Applications*, vol. 151, pp. 1–18, 2020.

[8] S. Lian and M. Tang, "API recommendation for mashup creation based on neural graph collaborative filtering," *Connection Science*, vol. 34, no. 1, pp. 124–138, 2022.

[9] W. Gong, X. Zhang, Y. Chen, Q. He, A. Beheshti, and X. Xu, "DAWAR: Diversity-aware APIs recommendation for mashup creation based on correlation graph," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information (SIGIR)*, 2022, pp. 395–404.

[10] X. Wang, H. Wu, and C.-H. Hsu, "Mashup-oriented API recommendation via random walk on knowledge graph," *IEEE Access*, vol. 7, pp. 7651–7662, 2019.

[11] X. Wang, M. Xi, and J. Yin, "Functional and structural fusion based Web API recommendations in heterogeneous networks," in *IEEE International Conference on Web Services (ICWS)*, 2023, pp. 91–96.

[12] J. Mai, M. Tang, F. Xie, and L. Liao, "Third-party API recommendation based on heterogeneous hypergraph attention networks," in *IEEE International Conference on Web Services (ICWS)*, 2023, pp. 545–552.

[13] X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua, "Neural graph collaborative filtering," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2019, pp. 165–174.

[14] R. Mihalcea, C. Corley, and C. Strapparava, "Corpus-based and knowledge-based measures of text semantic similarity," in *21st National Conference on Artificial Intelligence (AAAI)*, 2006, pp. 775–780.

[15] X. Ye, H. Shen, X. Ma, R. Bunescu, and C. Liu, "From word embeddings to document similarities for improved information retrieval in software engineering," in *IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 404–415.

[16] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 855–864.

[17] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *Computer Science*, 2013.

[18] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2014, pp. 701–710.

[19] Y. Hu, F. Xiong, S. Pan, X. Xiong, L. Wang, and H. Chen, "Bayesian personalized ranking based on multiple-layer neighborhoods," *Information Sciences*, vol. 542, pp. 156–176, 2021.

[20] G. Kang, J. Liu, B. Cao, and M. Cao, "NAFM: Neural and attentional factorization machine for Web API recommendation," in *IEEE International Conference on Web Services (ICWS)*, 2020, pp. 330–337.

[21] G. Kang, J. Liu, Y. Xiao, B. Cao, Y. Xu, and M. Cao, "Neural and attentional factorization machine-based Web API recommendation for mashup development," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4183–4196, 2021.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:33:12 UTC from IEEE Xplore. Restrictions apply.